

Python - Nested Dictionaries

Nested Dictionaries

Nested dictionaries in Python refer to dictionaries that are stored as values within another **dictionary**. In other words, a dictionary can contain other dictionaries as its values, forming a hierarchical or nested structure.

Nested dictionaries can be modified, updated, or extended in the same way as regular dictionaries. You can add, remove, or update key-value pairs at any level of the nested structure.

Creating a Nested Dictionary in Python

We can create a nested dictionary in Python by defining a dictionary where the values of certain keys are themselves dictionaries. This allows for the creation of a hierarchical structure where each key-value pair represents a level of nested information. This can be achieved in several ways –

Example: Direct Assignment

In this approach, we can directly assign dictionaries as values to outer keys within a single dictionary definition –

```
# Define the outer dictionary
nested_dict = {
    "outer_key1": {"inner_key1": "value1", "inner_key2": "value2"},
    "outer_key2": {"inner_key3": "value3", "inner_key4": "value4"}
}
print(nested_dict)
```

Example: Using a Loop

With this method, an empty outer dictionary is initialized, and then populated with dictionaries as values using a loop to define nested dictionaries –

```
# Define an empty outer dictionary
nested_dict = {}

# Add key-value pairs to the outer dictionary
outer_keys = ["outer_key1", "outer_key2"]
for key in outer_keys:
    nested_dict[key] = {"inner_key1": "value1", "inner_key2": "value2"}
print(nested_dict)
```

Adding Items to a Nested Dictionary in Python

Once a nested dictionary is created, we can add items to it by accessing the specific nested dictionary using its key and then assigning a new key-value pair to it.

Example

In the following example, we are defining a nested dictionary "students" where each key represents a student's name and its value is another dictionary containing details about the student.

Then, we add a new key-value pair to Alice's nested dictionary and add a new nested dictionary for a new student, Charlie –

```
# Initial nested dictionary
students = {
    "Alice": {"age": 21, "major": "Computer Science"},
    "Bob": {"age": 20, "major": "Engineering"}
}

# Adding a new key-value pair to Alice's nested dictionary
students["Alice"]["GPA"] = 3.8
```

```
# Adding a new nested dictionary for a new student
students["Charlie"] = {"age": 22, "major": "Mathematics"}

print(students)
```

It will produce the following output –

```
{'Alice': {'age': 21, 'major': 'Computer Science', 'GPA': 3.8}, 'Bob': {'age': 20, 'major': 'Engineering', 'GPA': 3.5}, 'Charlie': {'age': 22, 'major': 'Mathematics', 'GPA': 3.7}}
```

Accessing Items of a Nested Dictionary in Python

Accessing items of a nested dictionary in Python refers to retrieving values stored within the nested structure by using a series of keys. Each key corresponds to a level in the hierarchy of the dictionary.

We can achieve this through direct indexing with square brackets or by using the `get()` method

Example: Using Direct Indexing

In this approach, we access values in a nested dictionary by specifying each key in a sequence of square brackets. Each key in the sequence refers to a level in the nested dictionary, progressing one level deeper with each key –

```
# Define a nested dictionary
students = {
    "Alice": {"age": 21, "major": "Computer Science"},
    "Bob": {"age": 20, "major": "Engineering"},
    "Charlie": {"age": 22, "major": "Mathematics"}
}

# Access Alice's major
alice_major = students["Alice"]["major"]
print("Alice's major:", alice_major)

# Access Bob's age
bob_age = students["Bob"]["age"]
print("Bob's age:", bob_age)
```

Following is the output of the above code –

```
Alice's major: Computer Science
Bob's age: 20
```

Example: Using the get() Method

The get() method is used to fetch the value associated with the specified key. If the key does not exist, it returns a default value (which is None if not specified) –

```
# Define a nested dictionary
students = {
    "Alice": {"age": 21, "major": "Computer Science"},
    "Bob": {"age": 20, "major": "Engineering"},
    "Charlie": {"age": 22, "major": "Mathematics"}
}

# Access Alice's major using .get()
alice_major = students.get("Alice", {}).get("major", "Not Found")
print("Alice's major:", alice_major)

# Safely access a non-existing key using .get()
dave_major = students.get("Dave", {}).get("major", "Not Found")
print("Dave's major:", dave_major)
```

Output of the above code is as follows –

```
Alice's major: Computer Science
Dave's major: Not Found
```

Deleting a Dictionary from a Nested Dictionary

We can delete dictionaries from a nested dictionary by using the **del** keyword. This keyword allows us to remove a specific key-value pair from the nested dictionary.

Example

In the following example, we delete the nested dictionary for "Bob" from "students" dictionary using the del statement –

```
# Define a nested dictionary
students = {
    "Alice": {"age": 21, "major": "Computer Science"},
    "Bob": {"age": 20, "major": "Engineering"},
    "Charlie": {"age": 22, "major": "Mathematics"}
}

# Delete the dictionary for Bob
del students["Bob"]

# Print the updated nested dictionary
print(students)
```

We get the output as shown below –

```
{'Alice': {'age': 21, 'major': 'Computer Science'}, 'Charlie': {'age': 22, 'major': 'Mathematics'}}
```

Iterating Through a Nested Dictionary in Python

Iterating through a nested dictionary refers to looping through the keys and values at each level of the dictionary. This allows you to access and manipulate items within the nested structure.

We can iterate through a nested dictionary by using nested loops. The outer loop iterates over the keys and values of the main dictionary, while the inner loop iterates over the keys and values of the nested dictionaries.

Example

In this example, we are iterating through the "students" dictionary, retrieving each student's name and their corresponding details by iterating through the nested dictionaries –

```
# Defining a nested dictionary
students = {
    "Alice": {"age": 21, "major": "Computer Science"},
    "Bob": {"age": 20, "major": "Engineering"},
    "Charlie": {"age": 22, "major": "Mathematics"}
}

# Iterating through the Nested Dictionary:
for student, details in students.items():
    print(f"Student: {student}")
    for key, value in details.items():
        print(f"    {key}: {value}")
```

The output obtained is as shown below –

```
Student: Alice
    age: 21
    major: Computer Science
Student: Bob
    age: 20
    major: Engineering
Student: Charlie
    age: 22
    major: Mathematics
```

TOP TUTORIALS

Python Tutorial

Java Tutorial

C++ Tutorial

C Programming Tutorial

C# Tutorial

PHP Tutorial

R Tutorial

HTML Tutorial

CSS Tutorial

JavaScript Tutorial

[SQL Tutorial](#)

TRENDING TECHNOLOGIES

[Cloud Computing Tutorial](#)

[Amazon Web Services Tutorial](#)

[Microsoft Azure Tutorial](#)

[Git Tutorial](#)

[Ethical Hacking Tutorial](#)

[Docker Tutorial](#)

[Kubernetes Tutorial](#)

[DSA Tutorial](#)

[Spring Boot Tutorial](#)

[SDLC Tutorial](#)

[Unix Tutorial](#)

CERTIFICATIONS

[Business Analytics Certification](#)

[Java & Spring Boot Advanced Certification](#)

[Data Science Advanced Certification](#)

[Cloud Computing And DevOps](#)

[Advanced Certification In Business Analytics](#)

[Artificial Intelligence And Machine Learning](#)

[DevOps Certification](#)

[Game Development Certification](#)

[Front-End Developer Certification](#)



Chapters ▾

Categories ≡

COMPILERS & EDITORS

[Online Java Compiler](#)

[Online Python Compiler](#)

[Online Go Compiler](#)

[Online C Compiler](#)

[Online C++ Compiler](#)

[Online C# Compiler](#)

[Online PHP Compiler](#)

[Online MATLAB Compiler](#)

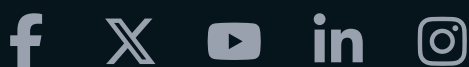
[Online Bash Terminal](#)

[Online SQL Compiler](#)

[Online Html Editor](#)

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |

[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.